

Agentic AI

Agent-to-Agent Protocol (A2A)

Agenda

01

Multi-Agent AI Fundamentals

Understanding autonomous AI systems and multi-agent architectures

03

Ecosystem and Comparisons

Industry landscape, A2A vs. MCP vs. APIs

02

A2A Protocol Deep Dive

Principles, mechanics, and architecture patterns

04

Security and Governance

Practical takeaways for enterprise deployment

From Single Agents to Multi-Agent Systems



Early Pattern

One agent with tools, centralized orchestration



Scaling Pattern

Multiple specialized agents with distinct roles



Benefits

Specialization, parallelism, robustness, structure



Cost: Coordination complexity requires clear interaction semantics



Why Agent-to-Agent Interaction Matters

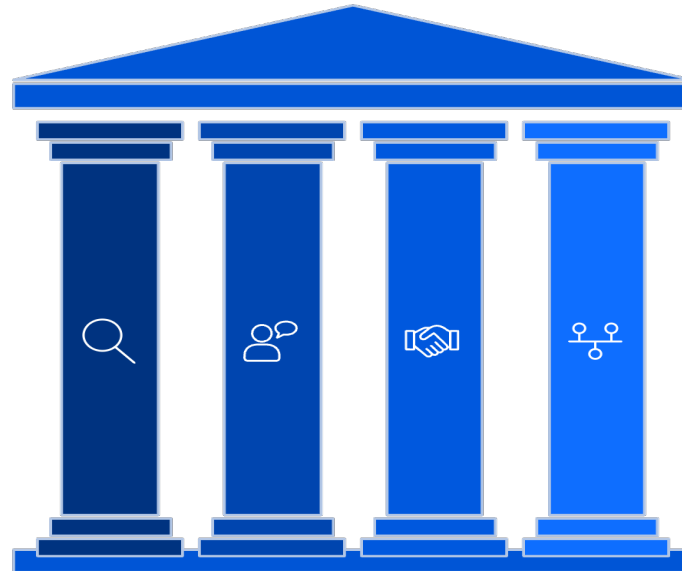
- Central orchestration becomes a bottleneck and single point of failure as systems scale.
- A2A protocols make multi-agent systems composable and interoperable, enabling agents to collaborate without tight coupling.

Discovery

Identify agents and capabilities

Negotiation

Propose, accept, reject, modify offers



Communication

Exchange intent and contextual data

Task State

Track long-running progress and status

A2A Overview

- **Agent2Agent (A2A) protocol**, developed by Google (now hosted by the Linux Foundation), is an open standard to enable **AI agents to interoperate** across platforms, vendors, and frameworks.
- The goal is to let autonomous agents **communicate, exchange info, and coordinate actions** securely in multi-agent ecosystems—increasing productivity and automation.

Partners Contributing to the Agent2Agent Protocol

Google Cloud

Partners contributing to the Agent2Agent protocol



Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

A2A Key Design Principles

- Agent2Agent (A2A) is an open protocol for AI agent interoperability, enabling agents from different builders, frameworks, or environments to collaborate.

Secure by Default

- Enterprise-grade authentication and authorization patterns

Long-Running Tasks

- Task lifecycle management with streaming progress updates

Modality-Agnostic

- Support for text and richer artifact exchange

Embrace Agentic Capabilities

- Supports unstructured communication without shared memory.

Built on Existing Standards

- Uses HTTP, SSE, JSON-RPC for easier integration.

Core Concepts: A2A Mechanics

Capability Discovery

Agents advertise their functions through Agent Card concept.

Task Lifecycle

Request →
accept/negotiate →
progress updates →
completion

Artifacts

Structured outputs like documents, links, summaries, and media can be exchanged.

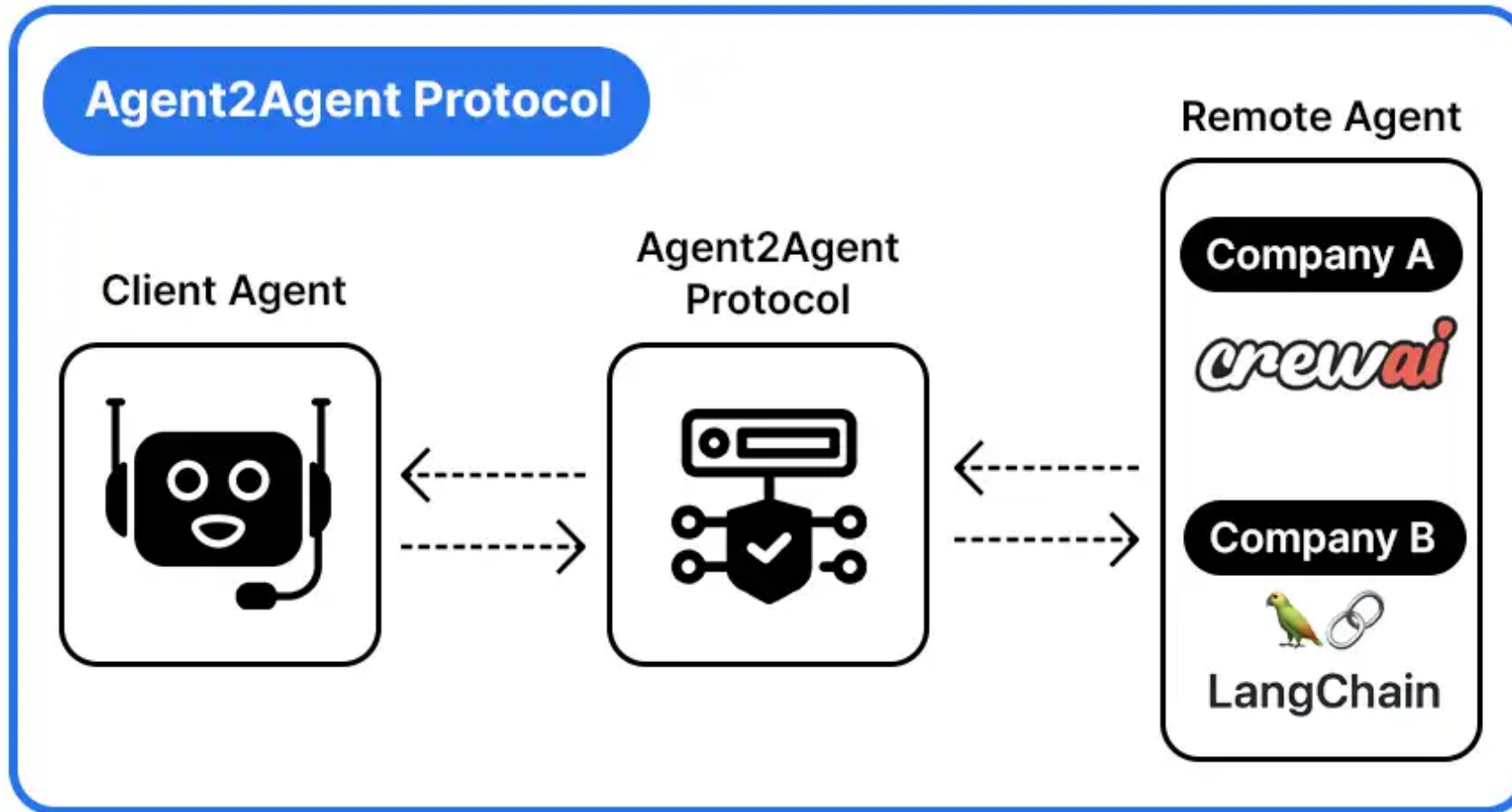
Operational Fit

Supports distributed, long-running workflows with incremental results.

Why A2A Matters for Agentic AI

- **Interoperability:**
 - Agents can collaborate across vendors and stacks.
- **Decentralized Coordination:**
 - Peer collaboration reduces orchestration bottlenecks.
- **Composable Workflows:**
 - Organizations can assemble agent “teams” for end-to-end processes.
- **Reduced Silos:**
 - Capabilities become discoverable and reusable rather than bespoke integrations.

How It Works

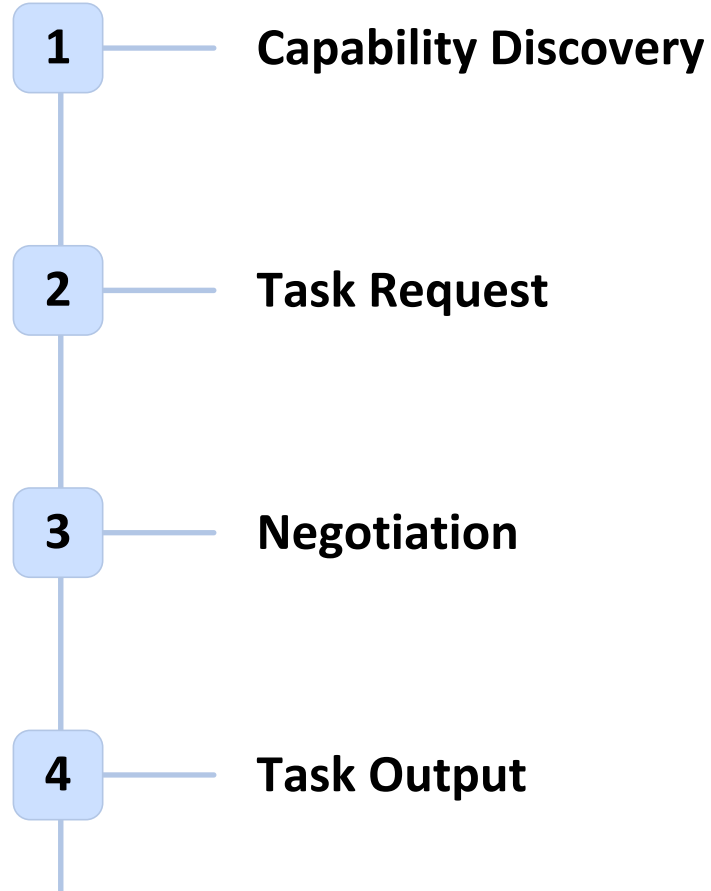


[Image Source](#)

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

A2A Message Flow Architecture



A2A Ecosystem Roles

A2A standardizes agent-to-agent collaboration while tool access remains implementation-specific.

- Users define intent, orchestrators plan and broker tasks.
- Specialist agents execute using enterprise systems and external services.

A2A vs. MCP vs. Traditional APIs

Dimension	Traditional APIs	MCP	A2A
Primary interaction	App → Service	Agent → Tool/Data	Agent → Agent
Autonomy	None	Caller autonomous	Both autonomous
Style	Request/response	Context + tool calls	Negotiated, bi-directional
Discovery	Human docs	Tool manifests	Agent Cards
Long tasks	Usually stateless	Session context	Task lifecycle + progress
Negotiation	None	None	Accept/reject/modify

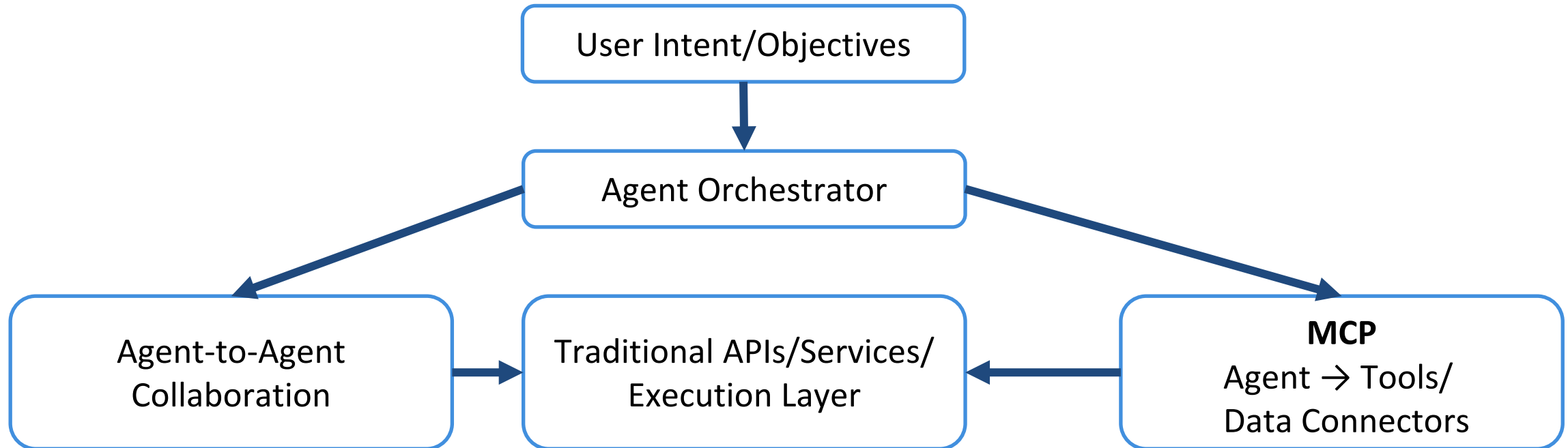
Key Insight:

- APIs scale software.
- MCP scales tool access.
- A2A scales coordinated intelligence.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited

This file is meant for personal use by ali@zarreh.ai only.
Sharing or publishing the contents in part or full is liable for legal action.

Interoperability Stack



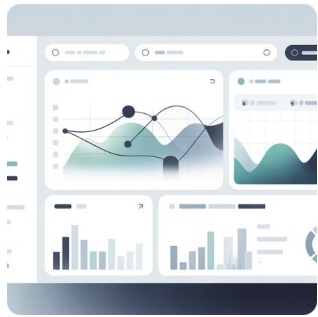
Key Insight:

- Many newly developed systems use all three services.
 - A2A for Peer Collaboration
 - MCP for tool access
 - APIs for execution

Industry Ecosystem

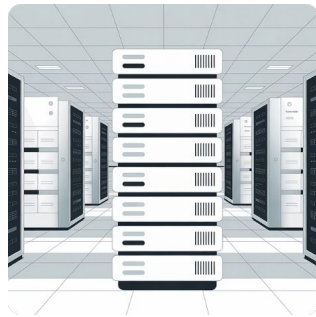
Adoption Signals

A2A is positioned as an **open interoperability layer** for agents across enterprise and platform ecosystems.



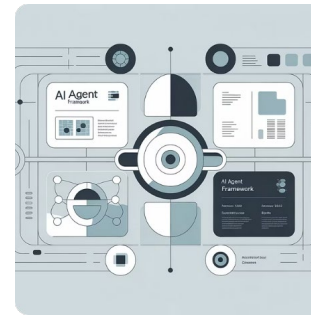
Enterprise Software

Workflow platforms
integrating A2A
capabilities



Knowledge Systems

Databases and information
repositories



Agent Frameworks

Core platforms enabling
agent development



System Integrators

Consultancies driving
implementation

<https://github.com/a2aproject/A2A>

Key Distinction: A2A vs. Tool Use

Tool Use

Agent calls a passive endpoint with deterministic interface and no negotiation.

A2A Interaction

- Agent communicates with an autonomous peer who can refuse, renegotiate, or delegate onward.
- Responsibility and accountability need explicit semantics. Coordination becomes socio-technical with roles, incentives, and governance.

Security, Governance and Next Steps

Critical Challenges

- Malicious or compromised agents
- Inappropriate delegation and overreach
- Emergent feedback loops
- Unclear accountability

Mitigation

Strong identity, capability scoping, policy agents, observability

Operational Best Practices

Treat as Distributed Systems

Explicit contracts, failure semantics, staged rollout

Use Protocols Strategically

A2A for collaboration, MCP for tools, APIs for execution

Governance First

Interoperability requires governance to keep pace

Learn more: [Google Developers Blog - A2A Protocol](#)

Operational Takeaways

Treat as Distributed Systems

- Explicit contracts and failure semantics
- Observability and evaluation
- Staged rollout and guardrails

Use Protocols to Reduce Integration

- A2A for agent collaboration
- MCP for tool/data integration
- APIs for execution

Governance is Critical

Interoperability is a force multiplier only if governance keeps pace with autonomy

Key Takeaways

Agentic AI Scales

From single agents to teams of specialized agents working together

A2A Enables Collaboration

Cross-agent interoperability through discovery, negotiation, and artifact exchange

MCP Standardizes Access

Unified tool and data integration for agent ecosystems

APIs Remain Foundational

Traditional APIs are essential for execution but insufficient alone

Security Is First-Class

Governance requirements increase as agent autonomy grows